

INTERACTIVE QUIZ APPLICATION

A Modular Python CLI Assessment and Performance Tracking System

Submitted By: Gracie Bagora

Reg. No: 26BH10004

Branch: CSE (Health Informatics)

Course Code: CSEI021

VT Bhopal University

Table of Contents

1. Introduction
2. Aim of the Project
3. Problem Statement & Scope
4. Software & System Requirements
5. System Architecture & Module Design
6. System Design Diagrams (UML)
7. Development & Implementation Details
8. Project Execution Screenshots
9. Testing Strategy & Verification
10. Challenges & Key Learnings
11. Conclusion & Future Scope
12. Appendix – Complete Source Code

1. Introduction

Tools for self-assessment are valuable for assessing domain knowledge and learning retention. Python Interactive Quiz Application CLI project is an application developed in Python using modular object-oriented design principles. The system lets users answer multiple choice questions. It safely validates inputs. It calculates the score, maintains performance logs locally with JSON persistence. It generates the performance summary report.

2. Aim of the Study

The main objective is to create a terminal-native CLI assessment tool that:

- Provides interactive quizzes on several topics
- Implements validation loops to handle errors
- Stores user ratings and history in local storage JSON
- Creates a summary and historical performance analysis
- Follows standards for modular design, automated testing and version control git.

3. Problem Statement & Scope

- Problem Statement: Learners lack ready solutions to test their subject knowledge and track performance trends on their machines without a heavy desktop GUI and server configuration.
- Scope: The scope of the application is user sessions, dynamic question rendering in real time, scoring in real time, persistence in json file, and summary analytics through terminal menu.

4. Software & System Requirements

□ Language & Runtime: Python 3.x

□ Standard Libraries: json, os, unittest

□ Data Storage: JSONFile Persistence (results.json)

□ Version Control: Git & GitHub

(github.com/graciebagora/quiz_app)

(https://github.com/graciebagora/quiz_app)

□ Development Environment: Visual Studio Code / Command Prompt

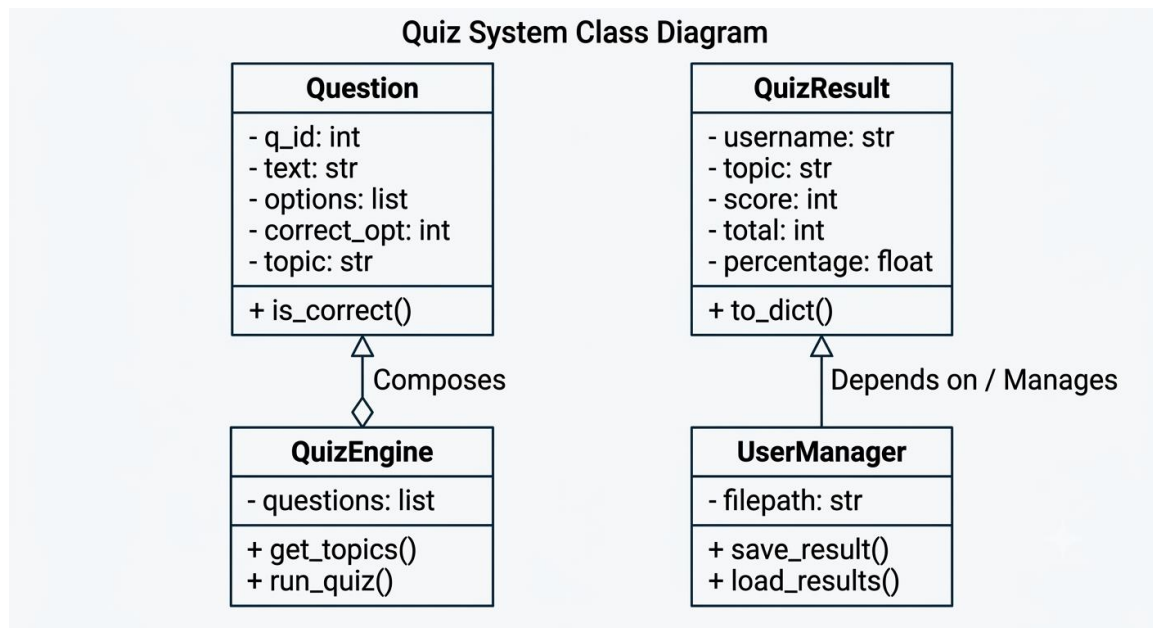
5. System Architecture & Module Design

The project uses a decoupled, modular architecture split into distinct files:

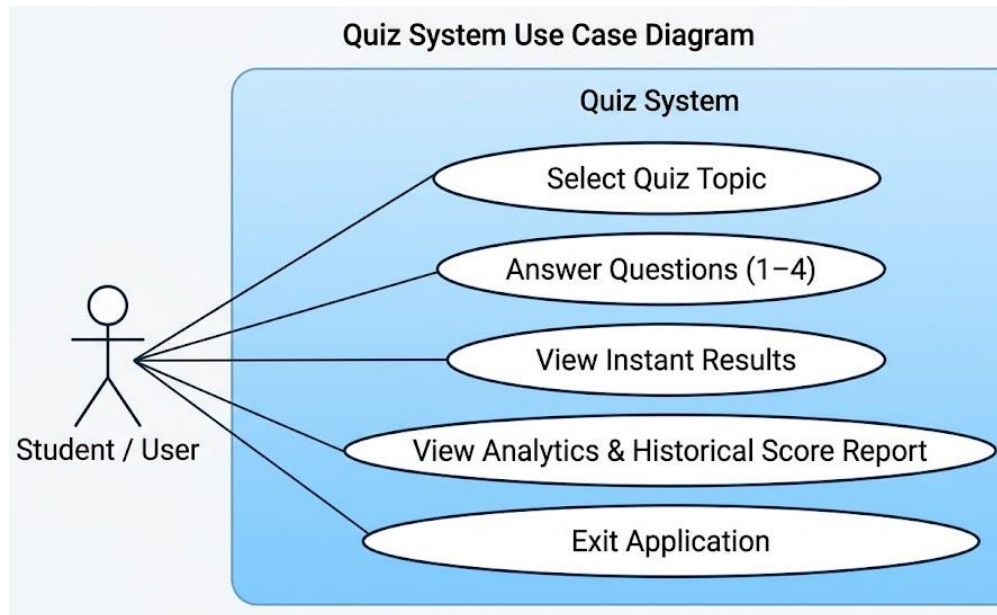
```
quiz_app/
├── main.py          # Interactive CLI driver menu
├── README.md        # Setup and testing instructions
├── statement.md     # Problem statement and features
├── results.json     # Auto-generated JSON storage
├── src/
│   ├── __init__.py  # Package marker
│   ├── models.py    # Data entities (Question, QuizResult)
│   ├── user_manager.py # JSON storage I/O
│   ├── quiz_engine.py # Quiz logic and input validation
│   └── analytics.py  # Analytics and summary reporting
└── tests/
    ├── __init__.py  # Package marker
    └── test_quiz.py  # Automated unit tests
```

6. System Design Diagrams(UM)

A Class Diagram



B Use Case Diagram



7. Development & Implementation Details

□ Domain Models (src/models.py):

Defines Question and QuizResult objects with encapsulated evaluation methods.

□ Data Persistence (src/user_manager.py):

Uses standard json operations to read and write user results to results.json.

□ Quiz Engine (src/quiz_engine.py):

Handles choice prompts, checks responses, and uses try/except loops to intercept non-numeric or out-of-bounds user inputs.

□ Analytics (src/analytics.py):

Parses stored JSON objects to compute total quizzes taken, overall average score percentage, and topic breakdowns.

8. Project Execution Screenshots

1. Main Menu Output

```
C:\Windows\System32\cmd.exe - python main.py
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>cd "C:\Users\Gracie Bagora\quiz_app"

C:\Users\Gracie Bagora\quiz_app>python main.py
=====
Welcome to the Quiz Application
=====
Enter your username: abc12

--- Main Menu ---
1. Take a Quiz
2. View Performance Report
3. Exit
Select an option (1-3): _
```

2. Quiz Execution & Input Validation

 C:\Windows\System32\cmd.exe - python main.py

```
C:\Windows\system32>cd "C:\Users\Gracie Bagora\quiz_app"
```

```
C:\Users\Gracie Bagora\quiz_app>python main.py
```

```
=====
```

```
Welcome to the Quiz Application
```

```
=====
```

```
Enter your username: abc12
```

```
--- Main Menu ---
```

```
1. Take a Quiz
```

```
2. View Performance Report
```

```
3. Exit
```

```
Select an option (1-3): 1
```

```
Available Topics:
```

```
1. Python
```

```
2. Data Structures
```

```
3. General CS
```

```
Select topic number: 1
```

```
--- Starting Quiz: Python ---
```

```
Q1: What is the output of print(type([]))?
```

```
1. <class 'tuple'>
```

```
2. <class 'list'>
```

```
3. <class 'dict'>
```

```
4. <class 'set'>
```

```
Your choice (1-4): ab
```

```
Invalid input! Please enter a number.
```

```
Your choice (1-4):
```

3. Analytics Output:

```
Quiz complete! Score: 2/3

--- Main Menu ---
1. Take a Quiz
2. View Performance Report
3. Exit
Select an option (1-3): 2

=== Performance Report for abc12 ===
Topic: Python | Score: 4/4 (100.0%)
Topic: General CS | Score: 2/3 (66.7%)
Topic: Data Structures | Score: 2/3 (66.7%)
-----
Total Quizzes Taken: 3
Average Percentage: 77.78%
```

4. Unit Test Output

```
C:\Users\Gracie Bagora\quiz_app>python -m unittest discover -s tests -p "test_*.py"
..
-----
Ran 2 tests in 0.001s

OK
```

9. Testing Strategy & Verification

Automated testing is implemented using Python's built-in unittest framework in tests/test_quiz.py.

- Tested Components:
 - Correct option evaluation in Question.is_correct().
 - Score percentage calculation in QuizResult.
- Execution Command:
python -m unittest discover -s tests -p "test_*.py".
- Test Outcome: All unit tests run successfully with zero errors.

10. Challenges & Key Learnings

- Challenges: Preventing terminal crashes caused by invalid user inputs during quiz selection and managing remote branch synchronization during initial git push operations
- Learnings: Applied Object-Oriented Programming (OOP) concepts, JSON data management, CLI input validation, automated unit testing, and Git version control workflows

11. Conclusion & Future Scope

□ Conclusion: The Quiz Application demonstrates a clean, modular Python CLI system meeting all technical and documentation criteria.

□ Future Scope:

- Adding timed quiz modes
- Loading questions from remote REST APIs
- Adding user authentication and password protection

12. Appendix – Source Code

-src/models.py

```
"""Data models for Questions and Quiz Results."""

class Question:
    def __init__(self, q_id: int, text: str, options: list,
correct_option: int, topic: str):
        self.q_id = q_id
        self.text = text
        self.options = options
        self.correct_option = correct_option
        self.topic = topic

    def is_correct(self, user_choice: int) -> bool:
        return user_choice == self.correct_option

class QuizResult:
    def __init__(self, username: str, topic: str, score: int, total:
int):
        self.username = username
        self.topic = topic
        self.score = score
```

```

        self.total = total
        self.percentage = (score / total) * 100 if total > 0 else 0.0

    def to_dict(self) -> dict:
        return {
            "username": self.username,
            "topic": self.topic,
            "score": self.score,
            "total": self.total,
            "percentage": self.percentage
        }

```

-src/user_manager.py

```

"""Module 1: User Management & Data Storage"""
import json
import os

class UserManager:
    def __init__(self, filepath="results.json"):
        self.filepath = filepath
        self._ensure_file_exists()

    def _ensure_file_exists(self):
        if not os.path.exists(self.filepath):
            with open(self.filepath, "w") as f:
                json.dump([], f)

    def save_result(self, quiz_result):
        data = self.load_all_results()
        data.append(quiz_result.to_dict())
        with open(self.filepath, "w") as f:
            json.dump(data, f, indent=4)

    def load_all_results(self) -> list:
        try:
            with open(self.filepath, "r") as f:
                return json.load(f)
        except (json.JSONDecodeError, FileNotFoundError):
            return []

```

-main.py

```

"""Main CLI Driver"""
from src.quiz_engine import QuizEngine
from src.user_manager import UserManager
from src.analytics import AnalyticsManager

```

```

def main():
    user_manager = UserManager()
    quiz_engine = QuizEngine()
    analytics = AnalyticsManager(user_manager)

    username = input("Enter your username: ").strip()
    if not username:
        username = "Guest"

    while True:
        print("\n=== QUIZ APPLICATION ===")
        print("1. Take a Quiz")
        print("2. View Performance Report")
        print("3. Exit")
        choice = input("Enter choice (1-3): ").strip()

        if choice == "1":
            topics = quiz_engine.get_topics()
            print("\nAvailable Topics:")
            for idx, t in enumerate(topics, 1):
                print(f"{idx}. {t}")
            try:
                t_choice = int(input("Select topic number: "))
                if 1 <= t_choice <= len(topics):
                    selected_topic = topics[t_choice - 1]
                    result = quiz_engine.run_quiz(username,
selected_topic)
                    user_manager.save_result(result)
                    print(f"\nQuiz Finished! Score:
{result.score}/{result.total} ({result.percentage:.1f}%)")
                else:
                    print("Invalid topic selection.")
            except ValueError:
                print("Invalid input! Please enter a number.")

        elif choice == "2":
            analytics.display_analytics(username)
        elif choice == "3":
            print("Goodbye!")
            break
        else:
            print("Invalid option. Please choose 1, 2, or 3.")

if __name__ == "__main__":
    main()

```

```

- src/quiz_engine.py:
"""Quiz Engine Module - Handles quiz execution and input
validation"""
from src.models import Question, QuizResult

class QuizEngine:
    def __init__(self):
        self.questions = [
            Question(1, "What is the output of print(type([]))?",
                ["<class 'tuple'>", "<class 'list'>",
                "<class 'dict'>", "<class 'set'>"], 2, "Python"),
            Question(2, "What is the time complexity of Binary
Search?",
                ["O(n)", "O(n^2)", "O(log n)", "O(1)"], 3,
                "Data Structures"),
            Question(3, "Which data structure follows FIFO (First
In First Out)?",
                ["Stack", "Queue", "Binary Tree", "Graph"],
                2, "Data Structures")
        ]

    def get_topics(self):
        return sorted(list(set(q.topic for q in self.questions)))

    def run_quiz(self, username, topic):
        topic_questions = [q for q in self.questions if q.topic
== topic]
        score = 0
        for q in topic_questions:
            print(f"\n{q.text}")
            for idx, opt in enumerate(q.options, 1):
                print(f"{idx}. {opt}")

            while True:
                choice = input("Your choice (1-4): ").strip()
                if choice.isdigit() and 1 <= int(choice) <=
len(q.options):
                    user_choice = int(choice)
                    break
                print("Invalid input! Please enter a number.")

            if q.is_correct(user_choice):
                print("Correct!")
                score += 1
            else:

```

```
        print(f"Wrong! Correct answer was option  
{q.correct_option}.")
```

```
    return QuizResult(username, topic, score,  
len(topic_questions))
```

- src/analytics.py

```
"""Analytics Module - Generates performance summaries"""
```

```
class AnalyticsManager:  
    def __init__(self, user_manager):  
        self.user_manager = user_manager  
  
    def display_analytics(self, username):  
        results = self.user_manager.load_all_results()  
        user_results = [r for r in results if r.get("username") == username]  
  
        if not user_results:  
            print(f"\nNo quiz history found for user '{username}'.")  
            return  
  
        print(f"\n=== Performance Report for {username} ===")  
        total_quizzes = len(user_results)  
        total_percentage = 0.0  
  
        for r in user_results:  
            topic = r.get("topic")  
            score = r.get("score")  
            total = r.get("total")  
            pct = r.get("percentage", 0.0)  
            total_percentage += pct  
            print(f"Topic: {topic} | Score: {score}/{total} ({pct:.1f}%")  
  
        avg_percentage = total_percentage / total_quizzes  
        print("-" * 35)  
        print(f"Total Quizzes Taken: {total_quizzes}")  
        print(f"Average Percentage: {avg_percentage:.2f}%")
```